Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

# Отчёт

по лабораторной работе №5
по курсу «Технологии глубоких нейронных сетей»
на тему «Классификация изображений»

Выполнил:
студент группы 23ВВИ2
Юров Д.М.

Принял:
Митрохин М.А.

Пенза 2026

**Цель работы:** научиться создавать собственные датасеты, изучить применение сверточных нейронных сетей для классификации изображений и освоить дообучение предобученной модели AlexNet на собственном датасете.

**Задание:**

1. Подготовка
   1) Откройте Anaconda Prompt или консоль команд, активируйте созданную для выполнения лабораторных работ среду с установленным PyTorch,
   2) Запустите IDE Spyder,
   3) Откройте файл «Lab5_cnn_classification.py»,
   4) Прочитайте комментарии и выполните код,
   5) После ознакомления с кодом выполните задание.
2. Задание 1
   1) Создайте свой собственный набор данных для классификации объектов не менее чем по 3 классам,
   2) Загрузите предобученную нейронную сеть из библиотеки PyTorch и обучите её на решение созданной задачи,

**Ход выполнения:**

1. Подготовка

1) Открыл Anaconda Prompt, активировал созданную для выполнения лабораторных работ среду с установленным PyTorch.

2) Запустил IDE Spyder.

3) Открыл файл «Lab5_cnn_classification.py», прочитал комментарии и выполнил код.

2. Сбор базы данных

1) Собрал базу «сырых» фотографий объектов классификации – Corvette c6, Mazda mx-5 «Miata», Volkswagen Passat b3, Porsche Cayenne 1 (рис. 1-2). В качестве источника фотографий использовал поисковую систему Яндекс.

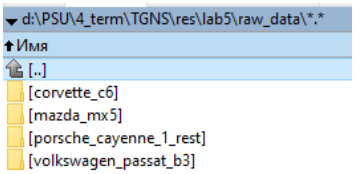



Рисунок 1 - папки "сырых" данных

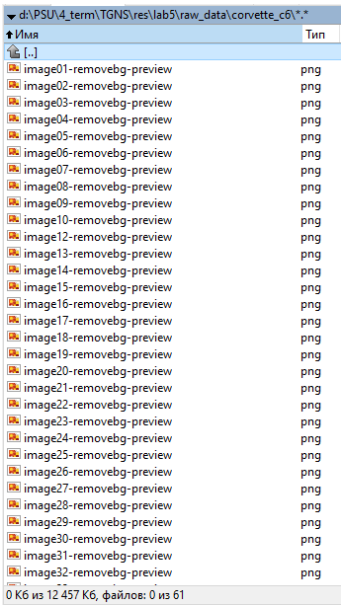


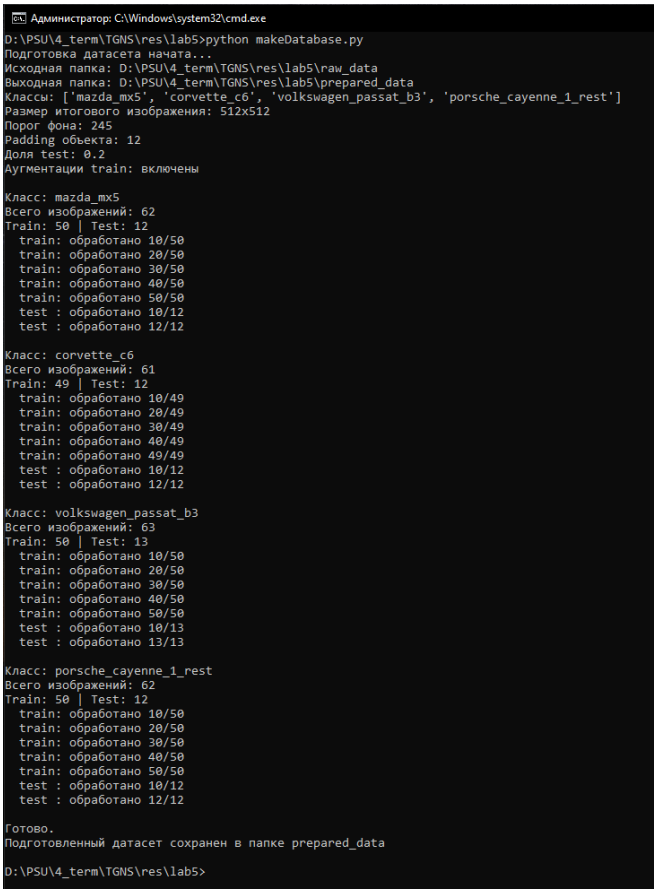

Рисунок 2 - файлы "сырых" данных

2) С помощью сайта «remove.bg» убрал фон со всех «сырых» фотографий.

3) Написал код скрипта, создающего базу данных из «сырых» фотографий (см. Приложение 2). Обработка фотографий происходит по следующему алгоритму:

- Определяется квадратное поле границ, в котором находится объект на фотографии,
- Создается 3 копии объекта – изначальный объект, с круговым смещением и отзеркаленный по вертикали,
- Объект размещается по центру, по бокам от объекта выставляются фиксированные поля.

Запустил скрипт (рис. 3).



```
Администратор: C:\Windows\system32\cmd.exe
D:\PSU\4_term\TGNS\res\lab5>python makeDatabase.py
Подготовка датасета начата...
Исходная папка: D:\PSU\4_term\TGNS\res\lab5\raw_data
Выходная папка: D:\PSU\4_term\TGNS\res\lab5\prepared_data
Классы: ['mazda_mx5', 'corvette_c6', 'volkswagen_passat_b3', 'porsche_cayenne_1_rest']
Размер итогового изображения: 512x512
Порог фона: 245
Padding объекта: 12
Доля test: 0.2
Аугментации train: включены

Класс: mazda_mx5
Всего изображений: 62
Train: 50 | Test: 12
  train: обработано 10/50
  train: обработано 20/50
  train: обработано 30/50
  train: обработано 40/50
  train: обработано 50/50
  test : обработано 10/12
  test : обработано 12/12

Класс: corvette_c6
Всего изображений: 61
Train: 49 | Test: 12
  train: обработано 10/49
  train: обработано 20/49
  train: обработано 30/49
  train: обработано 40/49
  train: обработано 49/49
  test : обработано 10/12
  test : обработано 12/12

Класс: volkswagen_passat_b3
Всего изображений: 63
Train: 50 | Test: 13
  train: обработано 10/50
  train: обработано 20/50
  train: обработано 30/50
  train: обработано 40/50
  train: обработано 50/50
  test : обработано 10/13
  test : обработано 13/13

Класс: porsche_cayenne_1_rest
Всего изображений: 62
Train: 50 | Test: 12
  train: обработано 10/50
  train: обработано 20/50
  train: обработано 30/50
  train: обработано 40/50
  train: обработано 50/50
  test : обработано 10/12
  test : обработано 12/12

Готово.
Подготовленный датасет сохранен в папке prepared_data

D:\PSU\4_term\TGNS\res\lab5>
```

Рисунок 3 - результат работы скрипта

Итоговые файлы базы данных заносятся в созданные каталоги классов в каталоге «prepared_data», причем они делятся на тренировочные данные и проверочные данные (рис. 4-6).

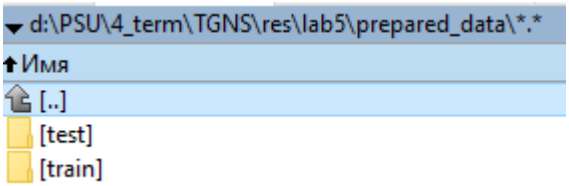



Рисунок 4 - подготовленные данные 1

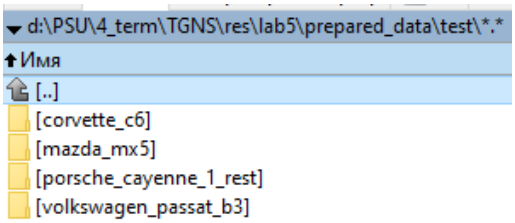



Рисунок 5 - подготовленные данные 2

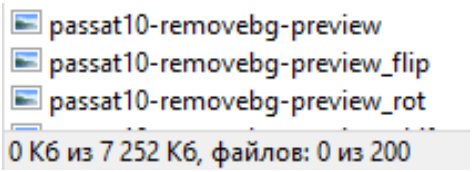



Рисунок 6 - подготовленные данные 3

То есть, код представляет собой алгоритм нормализации и аугментации изображений, необходимый для расширения базы данных, чтобы нейронная сеть не реагировала на незначительные изменения фотографии, такие как повороты.

3. Нейронные сети

1) На основе кода, содержащегося в файле «» создал собственную нейронную сеть (см. Приложение 1).

Вывод данных об устройстве и каталогах базы данных представлен на рисунке 7.

```
Устройство: cuda:0
Обучающий набор: ./prepared_data/train
Тестовый набор: ./prepared_data/test
```

Рисунок 7 - вывод 1

Вывод данных о классах представлен на рисунке 8.

```
Найденные классы: ['corvette_c6', 'mazda_mx5', 'porsche_cayenne_1_rest', 'volkswagen_passat_b3']
Ожидались: ['mazda_mx5', 'corvette_c6', 'volkswagen_passat_b3', 'porsche_cayenne_1_rest']
Первый элемент обучающего набора: ('./prepared_data/train\\corvette_c6\\image02-removebg-preview.jpg', 0)
Размер обучающего набора: 796
```

Рисунок 8 - вывод 2

Вывод размера batch представлен на рисунке 9.

```
In [5]: inputs, classes = next(iter(train_loader))
   ...: print('Размер batch:', inputs.shape) # 10 изображений, 3 канала (RGB), размер каждого 224x224 пикселя
   ...:
Размер batch: torch.Size([10, 3, 64, 64])
```

Рисунок 9 - вывод 3

Вывод плота с примерами тренировочных изображений представлен на рисунке 10.

Рисунок 10 - плот с выборкой тренировочных изображений

Обозначил, что обучение будет длиться 50 эпох и начал обучения своей сети (рис. 11-12).

```
   ...: print('Время обучения:', time.time() - t)
Эпоха 1 из 50 Шаг 0 Ошибка: 1.3916208744049072
Эпоха 1 из 50 Шаг 50 Ошибка: 1.3948161602020264
Эпоха 2 из 50 Шаг 0 Ошибка: 1.3781366348266602
Эпоха 2 из 50 Шаг 50 Ошибка: 1.3527189493179321
Эпоха 3 из 50 Шаг 0 Ошибка: 1.361788034439087
Эпоха 3 из 50 Шаг 50 Ошибка: 1.3769499063491821
Эпоха 4 из 50 Шаг 0 Ошибка: 1.393932580947876
Эпоха 4 из 50 Шаг 50 Ошибка: 1.4735820293426514
Эпоха 5 из 50 Шаг 0 Ошибка: 1.4797178506851196
Эпоха 5 из 50 Шаг 50 Ошибка: 1.1839510202407837
```

Рисунок 11 - начало обучения

```
Эпоха 49 из 50 Шаг 0 Ошибка: 0.0027228323742747307
Эпоха 49 из 50 Шаг 50 Ошибка: 0.0113583505153656
Эпоха 50 из 50 Шаг 0 Ошибка: 0.06457625329494476
Эпоха 50 из 50 Шаг 50 Ошибка: 0.024864576756954193
Время обучения: 278.89860558509827
```

Рисунок 12 - конец обучения

График уменьшения loss в процессе обучения представлен на рисунке 13.

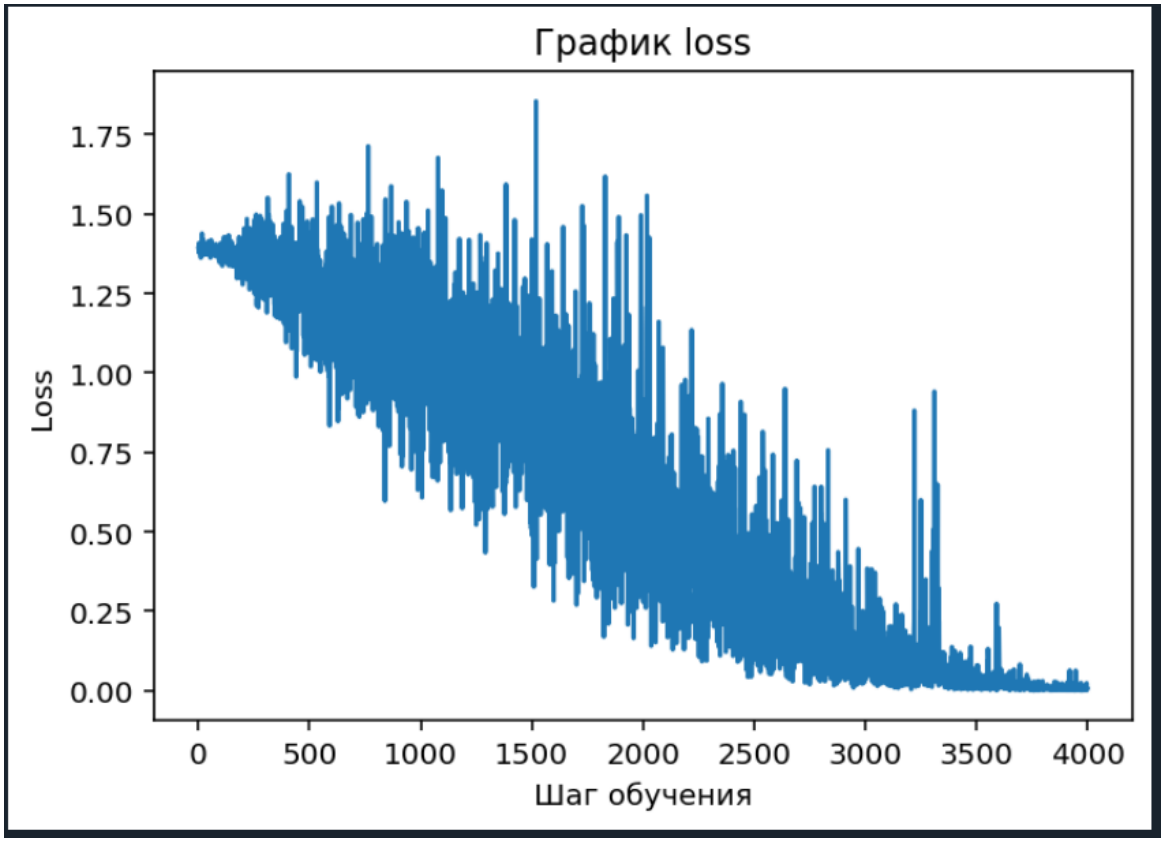



Рисунок 13 - график уменьшения loss своей сети

Посчитал точность своей модели: количество правильно классифицированных картинок (рис. 14).

```
In [10]: correct_predictions = 0
    ...: num_test_samples = len(test_dataset)
    ...:
    ...: with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    ...:     for images, labels in test_loader:
    ...:         images = images.to(device)
    ...:         labels = labels.to(device)
    ...:         pred = net(images) # делаем предсказание по пакету
    ...:         _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
    ...:         correct_predictions += (pred_class == labels).sum().item() # сравниваем ответ с правильной меткой
    ...:
    ...: print('Точность модели: ' + str(100 * correct_predictions / num_test_samples) + '%')
Точность модели: 61.224489795918366%
```

Рисунок 14 - точность своей модели

2) Загрузил модель AlexNet и вывел её архитектуру (рис. 15).

```
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to C:\Users\izada/.cache\torch\hub\checkpoints\alexnet-owt-7be5be79.pth
100.0%
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

Рисунок 15 - вывод архитектуры AlexNet

Проверил точность предсказания необученной AlexNet (рис. 16).

```
In [16]:
    ...: net.eval()
    ...: correct_predictions = 0
    ...: num_test_samples = len(test_dataset)
    ...:
    ...: with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    ...:     for images, labels in test_loader:
    ...:         images = images.to(device)
    ...:         labels = labels.to(device)
    ...:         pred = net(images) # делаем предсказание по пакету
    ...:         _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
    ...:         correct_predictions += (pred_class == labels).sum().item()
    ...:
    ...: print('Точность AlexNet до обучения: ' + str(100 * correct_predictions / num_test_samples) + '%')
    ...: # явно требуется обучение
    ...:
Точность AlexNet до обучения: 22.448979591836736%
```

Рисунок 16 - точность предсказания необученной AlexNet

Обучил AlexNet на своих данных (рис. 17-18).

```
AlexNet | Эпоха 1 из 10 | Шаг 0 | Ошибка: 0.7012918591499329
AlexNet | Эпоха 1 из 10 | Шаг 20 | Ошибка: 0.46239107847213745
AlexNet | Эпоха 1 из 10 | Шаг 40 | Ошибка: 0.49035653471946716
AlexNet | Эпоха 1 из 10 | Шаг 60 | Ошибка: 0.5885510444641113
```

Рисунок 17 - начало обучения AlexNet

```
AlexNet | Эпоха 10 из 10 | Шаг 0 | Ошибка: 0.13710950314998627
AlexNet | Эпоха 10 из 10 | Шаг 20 | Ошибка: 0.21435043215751648
AlexNet | Эпоха 10 из 10 | Шаг 40 | Ошибка: 0.2050984799861908
AlexNet | Эпоха 10 из 10 | Шаг 60 | Ошибка: 0.18696340918540955
Время обучения AlexNet: 61.797016620635986
```

Рисунок 18 - конец обучения AlexNet

Вывел график уменьшения loss в процессе обучения AlexNet (рис. 19).

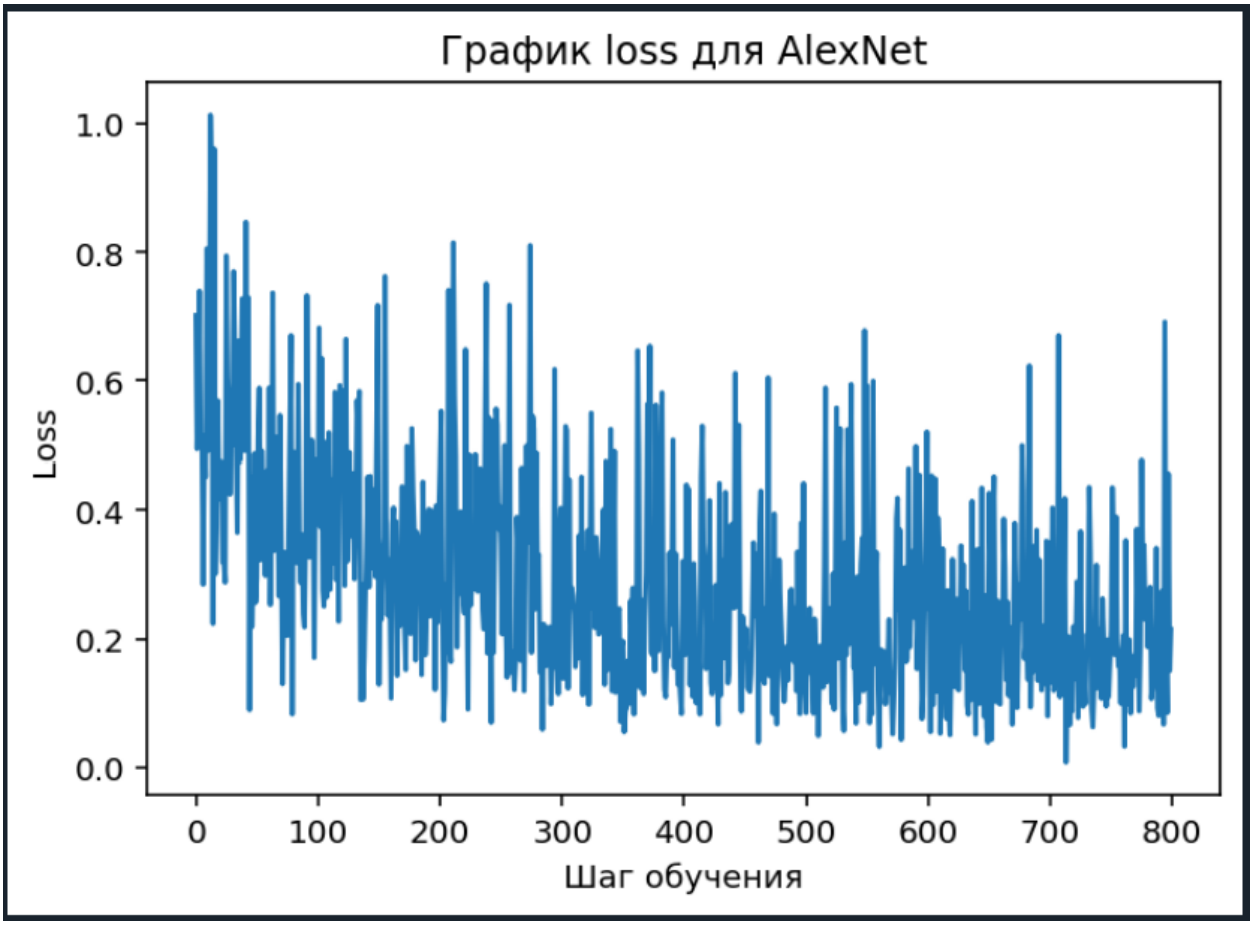



Рисунок 19 - график уменьшения loss AlexNet

AlexNet недостаточно обучилась на данных, запустил дополнительное обучение на 50 эпох (рис. 20-21).

```
AlexNet | Эпоха 1 из 50 | Шаг 0 | Ошибка: 0.03306347504258156
AlexNet | Эпоха 1 из 50 | Шаг 20 | Ошибка: 0.0887366533279419
AlexNet | Эпоха 1 из 50 | Шаг 40 | Ошибка: 0.32091599702835083
AlexNet | Эпоха 1 из 50 | Шаг 60 | Ошибка: 0.25342386960983276
AlexNet | Эпоха 2 из 50 | Шаг 0 | Ошибка: 0.29431286454200745
AlexNet | Эпоха 2 из 50 | Шаг 20 | Ошибка: 0.20499226450920105
AlexNet | Эпоха 2 из 50 | Шаг 40 | Ошибка: 0.10331179201602936
AlexNet | Эпоха 2 из 50 | Шаг 60 | Ошибка: 0.19181105494499207
```

Рисунок 20 - начало дополнительного обучения AlexNet

```
AlexNet | Эпоха 50 из 50 | Шаг 0 | Ошибка: 0.23472149670124054
AlexNet | Эпоха 50 из 50 | Шаг 20 | Ошибка: 0.26672548055648804
AlexNet | Эпоха 50 из 50 | Шаг 40 | Ошибка: 0.05851487070322037
AlexNet | Эпоха 50 из 50 | Шаг 60 | Ошибка: 0.23310992121696472
Время обучения AlexNet: 313.74652576446533
Out[21]: Text(0, 0.5, 'Loss')
```

Рисунок 21 - конец дополнительного обучения AlexNet

Вывел график уменьшения loss во время дополнительного обучения AlexNet (рис. 22).

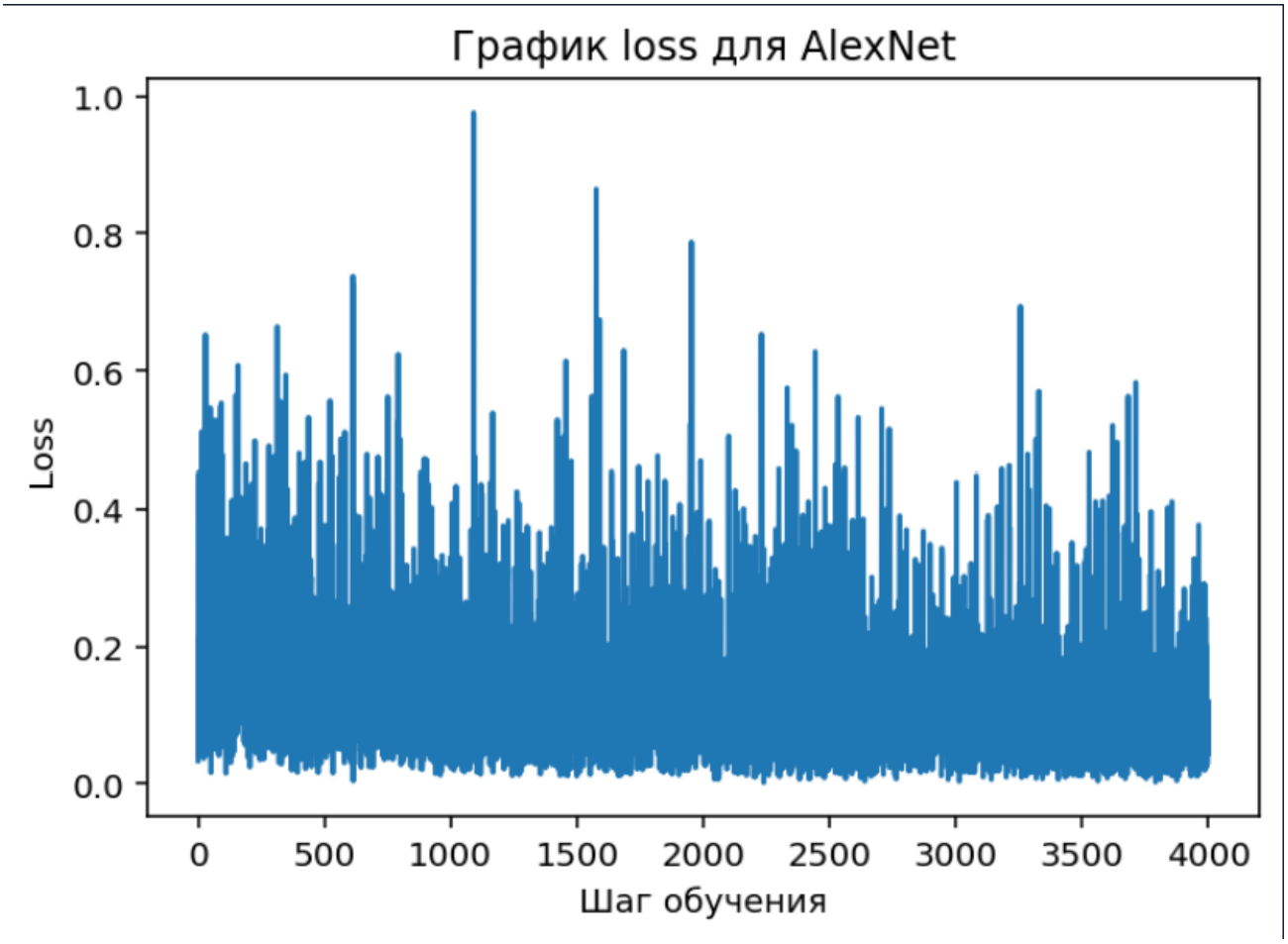



Рисунок 22 - уменьшение loss во время дополнительного обучения

Проверил точность предсказаний AlexNet после обучения (рис. 23).

```
In [22]: correct_predictions = 0
    ...: num_test_samples = len(test_dataset)
    ...:
    ...: with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    ...:     for images, labels in test_loader:
    ...:         images = images.to(device)
    ...:         labels = labels.to(device)
    ...:         pred = net(images) # делаем предсказание по пакету
    ...:         _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
    ...:         correct_predictions += (pred_class == labels).sum().item()
    ...:
    ...: print('Точность AlexNet после обучения: ' + str(100 * correct_predictions / num_test_samples) + '%')
    ...: # уже лучше
Точность AlexNet после обучения: 79.59183673469387%
```

Рисунок 23 - точность AlexNet после обучения

Сохранил обученную AlexNet (рис. 24).

```
In [23]:
    ...: torch.save(net.state_dict(), 'alexnet_cars_4classes.pth')
```

Рисунок 24 - сохранение обученной AlexNet

3) Вывел фотографии вместе с предсказаниями AlexNet об объектах, изображенных на фотографиях (рис. 25-34).

Предсказанный класс: mazda_mx5

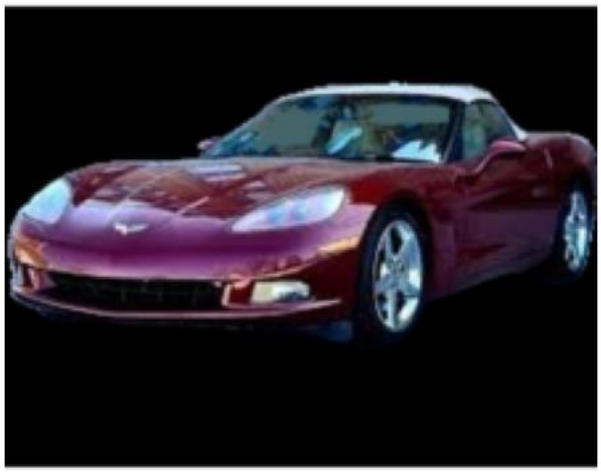


Рисунок 25 - предсказание 1

Предсказанный класс: corvette_c6

Рисунок 26 - предсказание 2

Предсказанный класс: corvette_c6

Рисунок 27 - предсказание 3

Предсказанный класс: mazda_mx5

Рисунок 28 - предсказание 4

Предсказанный класс: mazda_mx5

Рисунок 29 - предсказание 5

Предсказанный класс: corvette_c6

Рисунок 30 - предсказание 6

Предсказанный класс: corvette_c6

Рисунок 31 - предсказание 7

Предсказанный класс: volkswagen_passat_b3

Рисунок 32 - предсказание 8

Предсказанный класс: corvette_c6

Рисунок 33 - предсказание 9

Предсказанный класс: corvette_c6

Рисунок 34 - предсказание 10

**Вывод:** в ходе выполнения лабораторной работы научился создавать собственные датасеты, изучил применение сверточных нейронных сетей для классификации изображений и освоил дообучение предобученной модели AlexNet на собственном датасете.

# Приложение 1 – код «Lab5_cnn_classification_cars.py»

```
# -*- coding: utf-8 -*-
"""
1) mazda_mx5
2) corvette_c6
3) volkswagen_passat_b3
4) porsche_cayenne_1_rest
"""

import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'

import time
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
from torchvision.models import alexnet, AlexNet_Weights

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
print('Устройство:', device)

# Пути к данным
train_dir = './prepared_data/train'
test_dir = './prepared_data/test'
print('Обучающий набор:', train_dir)
print('Тестовый набор:', test_dir)

# Ожидаемые классы в наборе данных
expected_classes = [
    'mazda_mx5',
    'corvette_c6',
    'volkswagen_passat_b3',
    'porsche_cayenne_1_rest'
]

# 1. Простая сверточная сеть
# Так как простые сети работают с изображениями фиксированного размера,
# то наши изображения необходимо смасштабировать и преобразовать в тензор
data_transforms = transforms.Compose([
                        transforms.Resize(68),
                        transforms.CenterCrop(64),
                        transforms.ToTensor()])

# Создадим обучающий и тестовый наборы
train_dataset = torchvision.datasets.ImageFolder(root=train_dir,
                                                  transform=data_transforms)
test_dataset = torchvision.datasets.ImageFolder(root=test_dir,
                                                 transform=data_transforms)

# Посмотрим какие классы содержатся в наборе
class_names = train_dataset.classes
print('Найденные классы:', class_names)
print('Ожидались:', expected_classes)

# Список изображений можно получить следующим образом
train_set = train_dataset.samples
print('Первый элемент обучающего набора:', train_set[0])
```

```
print('Размер обучающего набора:', len(train_set))

batch_size = 10

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
                                    shuffle=True,  num_workers=2)


test_loader  = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size,
                                    shuffle=False, num_workers=2)


# Загрузим одну порцию данных
# Каждое обращение к DataLoader возвращает изображения и их классы
inputs, classes = next(iter(train_loader))
print('Размер batch:', inputs.shape) # 10 изображений, 3 канала (RGB), размер каждого 224x224
пикселя

# Построим сетку из изображений
img = torchvision.utils.make_grid(inputs, nrow=5) # метод делает сетку из картинок
img = img.numpy().transpose((1, 2, 0)) # для отображения через matplotlib
plt.figure()
plt.imshow(img)
plt.title('Примеры изображений из train')
plt.axis('off')


# Теперь можно переходить к созданию сети
# Для этого будем использовать как и ранее метод Sequential
# который объединит несколько слоев в один стек
class CnNet(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()
        self.layer1 = nn.Sequential(
        # первый сверточный слой с ReLU активацией и maxpooling-ом
            nn.Conv2d(3, 16, kernel_size=7, stride=1, padding=2), # 3 канала, 16 фильтров, размер
ядра 7
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # второй сверточный слой
        # количество каналов второго слоя равно количеству фильтров предыдущего слоя
        self.layer2 = nn.Sequential(
            nn.Conv2d(16, 32, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # третий сверточный слой
        # ядро фильтра от слоя к слою уменьшается
        self.layer3 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # классификационный слой имеет нейронов: количество фильтров * размеры карты признаков
        self.fc = nn.Linear(8 * 8 * 64, num_classes)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = self.layer3(out)
        out = out.reshape(out.size(0), -1) # флаттеринг
        out = self.fc(out)
        return out
```

```
# Количество классов
num_classes = len(class_names)

# создаем экземпляр сети
net = CnNet(num_classes).to(device)

# Задаем функцию потерь и алгоритм оптимизации
lossFn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(net.parameters(), lr=0.01)

# создаем цикл обучения и замеряем время его выполнения
num_epochs = 50
save_loss = []

t = time.time()
for epoch in range(num_epochs):
    net.train()
    for i, (images, labels) in enumerate(train_loader):
        images = images.to(device)
        labels = labels.to(device)

        # прямой проход
        outputs = net(images)
        # вычисление значения функции потерь
        loss = lossFn(outputs, labels)

        # Обратный проход (вычисляем градиенты)
        optimizer.zero_grad()
        loss.backward()
        # делаем шаг оптимизации весов
        optimizer.step()

        # сохраняем loss
        save_loss.append(loss.item())

        # выводим немного диагностической информации
        if i % 50 == 0:
            print('Эпоха ' + str(epoch + 1) + ' из ' + str(num_epochs) +
                  ' Шаг ' + str(i) + ' Ошибка: ' + str(loss.item()))

print('Время обучения:', time.time() - t)

# Посмотрим как уменьшался loss в процессе обучения
plt.figure()
plt.plot(save_loss)
plt.title('График loss')
plt.xlabel('Шаг обучения')
plt.ylabel('Loss')

# Посчитаем точность нашей модели: количество правильно классифицированных картинок
# поделенное на общее количество тестовых примеров
correct_predictions = 0
num_test_samples = len(test_dataset)

with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        pred = net(images) # делаем предсказание по пакету
        _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
        correct_predictions += (pred_class == labels).sum().item() # сравниваем ответ с правильной
меткой
```

```
print('Точность модели: ' + str(100 * correct_predictions / num_test_samples) + '%')

# сохраняем нашу модель в файл для дальнейшего использования
torch.save(net.state_dict(), 'CnNet_cars.ckpt')


# 2. Предобученная сеть AlexNet
# Так как сеть, которую мы планируем взять за базу натренирована на изображениях
# определенного размера, то наши изображения необходимо к ним преобразовать
alexnet_transforms = transforms.Compose([
                        transforms.Resize(256),
                        transforms.CenterCrop(224),
                        transforms.ToTensor(),
                        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                             std=[0.229, 0.224, 0.225] )
    ])

# Пересоздадим датасеты с учетом новых размеров и нормировки яркости
train_dataset = torchvision.datasets.ImageFolder(root=train_dir,
                                                  transform=alexnet_transforms)
test_dataset = torchvision.datasets.ImageFolder(root=test_dir,
                                                 transform=alexnet_transforms)

batch_size = 10

train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=batch_size,
                                           shuffle=True,
                                           num_workers=2)

test_loader = torch.utils.data.DataLoader(test_dataset,
                                          batch_size=batch_size,
                                          shuffle=False,
                                          num_workers=2)

# В качестве донора возьмем преобученную на ImageNet наборе сеть AlexNet
#      Список      доступных      предобученных      сетей      можно      посмотреть      тут
https://pytorch.org/vision/main/models.html
net = torchvision.models.alexnet(pretrained=True)


# можно посмотреть структуру этой сети
print(net)


# Так как веса feature_extractor уже обучены, нам нужно их "заморозить", чтобы
# быстрее научился наш классификатор
#  для этого отключаем у всех слоев (включая слои feature_extractor-а) градиенты
for param in net.parameters():
    param.requires_grad = False

# Меняем только последний слой классификатора под 4 класса

# Выходной слой AlexNet содержит 1000 нейронов (по количеству классов в ImageNet).
# Нам нужно его заменить на слой, содержащий только len(class_names) класса.

class_names = train_dataset.classes
num_classes = len(class_names)

new_classifier = net.classifier[:-1] # берем все слой классификатора кроме последнего
new_classifier.add_module('fc', nn.Linear(4096, num_classes))# добавляем последним слой с двумя
нейронами на выходе
net.classifier = new_classifier # меняем классификатор сети
```

```
net = net.to(device)

# проверим эффективность новой сети
net.eval()
correct_predictions = 0
num_test_samples = len(test_dataset)

with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        pred = net(images) # делаем предсказание по пакету
        _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
        correct_predictions += (pred_class == labels).sum().item()

print('Точность AlexNet до обучения: ' + str(100 * correct_predictions / num_test_samples) + '%')
# явно требуется обучение

# Перейдем к обучению.
# Зададим количество эпох обучения, функционал потерь и оптимизатор.
num_epochs = 50
lossFn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(net.classifier.parameters(), lr=0.001)

save_loss = []
#save_acc = []

# создаем цикл обучения и замеряем время его выполнения
t = time.time()
for epoch in range(num_epochs):
    net.train()
    correct_train = 0
    total_train = 0

    for i, (images, labels) in enumerate(train_loader):
        images = images.to(device)
        labels = labels.to(device)
        # прямой проход
        outputs = net(images)
        # вычисление значения функции потерь
        loss = lossFn(outputs, labels)
         # Обратный проход (вычисляем градиенты)

        optimizer.zero_grad()
        loss.backward()
        # делаем шаг оптимизации весов
        optimizer.step()
        save_loss.append(loss.item())

        #_, predicted = torch.max(outputs.data, 1)
        #total_train += labels.size(0)
        #correct_train += (predicted == labels).sum().item()

        # выводим немного диагностической информации
        if i % 20 == 0:
            print('AlexNet | Эпоха ' + str(epoch + 1) + ' из ' + str(num_epochs) +
                  ' | Шаг ' + str(i) + ' | Ошибка: ' + str(loss.item()))


print('Время обучения AlexNet:', time.time() - t)

# График loss
plt.figure()
```

```
plt.plot(save_loss)
plt.title('График loss для AlexNet')
plt.xlabel('Шаг обучения')
plt.ylabel('Loss')

# График accuracy
#plt.figure()
#plt.plot(save_acc)
#plt.title('График accuracy по эпохам для AlexNet')
#plt.xlabel('Эпоха')
#plt.ylabel('Accuracy, %')

# Еще раз посчитаем точность нашей модели (после обучения)
correct_predictions = 0
num_test_samples = len(test_dataset)

with torch.no_grad(): # отключим вычисление граиентов, т.к. будем делать только прямой проход
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        pred = net(images) # делаем предсказание по пакету
        _, pred_class = torch.max(pred.data, 1) # выбираем класс с максимальной оценкой
        correct_predictions += (pred_class == labels).sum().item()

print('Точность AlexNet после обучения: ' + str(100 * correct_predictions / num_test_samples) +
'%')
# уже лучше

# Сохраняем обученную модель
torch.save(net.state_dict(), 'alexnet_cars_4classes.pth')

# Реализуем отображение картинок и их класса, предсказанного сетью
inputs, classes = next(iter(test_loader))
pred = net(inputs.to(device))
_, pred_class = torch.max(pred.data, 1)

for i, j in zip(inputs, pred_class.cpu()):
    img = i.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img = std * img + mean
    img = np.clip(img, 0, 1)
    plt.figure()
    plt.imshow(img)
    plt.title('Предсказанный класс: ' + class_names[j])
    plt.axis('off')
    plt.show(block=False)
    plt.pause(2)
    plt.close()

print('Соответствие индексов классам:')
for idx, name in enumerate(class_names):
    print(idx, '->', name)

plt.show()
```

# Приложение 2 – код «makeDatabase.py»

```
import random
from pathlib import Path
from typing import List, Tuple

import numpy as np
from PIL import Image, ImageOps

# ----------------------------
# Настройки по умолчанию
# ----------------------------
RAW_ROOT = Path("raw_data")
OUTPUT_ROOT = Path("prepared_data")

CLASS_NAMES = [
    "mazda_mx5",
    "corvette_c6",
    "volkswagen_passat_b3",
    "porsche_cayenne_1_rest",
]

IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

# Белый фон в jpg обычно не идеально белый, поэтому считаем фоном
# все почти белые пиксели.
BACKGROUND_THRESHOLD = 245

# Отступы вокруг найденного объекта (в пикселях исходного изображения)
OBJECT_PADDING = 12

# Размер итогового квадратного изображения
OUTPUT_SIZE = 512

# Дополнительная внутренняя рамка внутри квадратного холста,
# чтобы машина не прилипала к краям
INNER_MARGIN = 24

# Доля изображений, идущих в test
TEST_RATIO = 0.2

# Зерно генератора случайных чисел для воспроизводимости
RANDOM_SEED = 42

# Создавать ли простые аугментации для train
CREATE_AUGMENTATIONS = True

# ----------------------------
# Служебные функции
# ----------------------------
def list_images(folder: Path) -> List[Path]:
    """Возвращает список изображений из папки."""
    if not folder.exists():
        return []
    return sorted(
        [p for p in folder.iterdir() if p.is_file() and p.suffix.lower() in IMAGE_EXTENSIONS]
    )


def ensure_dir(folder: Path) -> None:
    """Создает папку, если она не существует."""
```

```
    folder.mkdir(parents=True, exist_ok=True)


def crop_to_object(img: Image.Image,
                   bg_threshold: int = BACKGROUND_THRESHOLD,
                   padding: int = OBJECT_PADDING) -> Image.Image:
    """
    Находит ограничивающий прямоугольник объекта по отличию от белого фона
    и делает кроп с небольшим отступом.
    """
    rgb = img.convert("RGB")
    arr = np.array(rgb)

    # Маска: True там, где пиксель НЕ белый фон.
    # Берем пиксели, у которых хотя бы один канал заметно меньше порога.
    object_mask = np.any(arr < bg_threshold, axis=2)

    ys, xs = np.where(object_mask)

    # Если объект не найден, возвращаем исходное изображение
    if len(xs) == 0 or len(ys) == 0:
        return rgb

    left = max(int(xs.min()) - padding, 0)
    right = min(int(xs.max()) + padding + 1, rgb.width)
    top = max(int(ys.min()) - padding, 0)
    bottom = min(int(ys.max()) + padding + 1, rgb.height)

    return rgb.crop((left, top, right, bottom))


def paste_on_white_square(img: Image.Image,
                          output_size: int = OUTPUT_SIZE,
                          inner_margin: int = INNER_MARGIN) -> Image.Image:
    """
    Масштабирует объект с сохранением пропорций и помещает его
    на белый квадратный холст фиксированного размера.
    """
    img = img.convert("RGB")
    w, h = img.size

    max_w = output_size - 2 * inner_margin
    max_h = output_size - 2 * inner_margin
    if max_w <= 0 or max_h <= 0:
        raise ValueError("INNER_MARGIN слишком большой относительно OUTPUT_SIZE.")

    scale = min(max_w / w, max_h / h)
    new_w = max(1, int(round(w * scale)))
    new_h = max(1, int(round(h * scale)))

    resized = img.resize((new_w, new_h), Image.Resampling.LANCZOS)

    canvas = Image.new("RGB", (output_size, output_size), (255, 255, 255))
    x = (output_size - new_w) // 2
    y = (output_size - new_h) // 2
    canvas.paste(resized, (x, y))

    return canvas


def prepare_single_image(path: Path) -> Image.Image:
    """
    Полный конвейер подготовки одного изображения:
    открыть -> кроп по объекту -> квадратный белый холст.
```

```
    """
    with Image.open(path) as img:
        rgb = img.convert("RGB")
        cropped = crop_to_object(rgb)
        prepared = paste_on_white_square(cropped)
    return prepared


def save_image(img: Image.Image, path: Path) -> None:
    """Сохраняет изображение в JPG."""
    ensure_dir(path.parent)
    img.save(path, format="JPEG", quality=95)


def shift_image(img: Image.Image, dx: int = 0, dy: int = 0) -> Image.Image:
    """
    Сдвигает изображение на белом холсте.
    """
    src = img.convert("RGB")
    arr = np.array(src)

    object_mask = np.any(arr < BACKGROUND_THRESHOLD, axis=2)
    ys, xs = np.where(object_mask)

    if len(xs) == 0 or len(ys) == 0:
        return src.copy()

    left = int(xs.min())
    right = int(xs.max()) + 1
    top = int(ys.min())
    bottom = int(ys.max()) + 1

    obj = src.crop((left, top, right, bottom))
    canvas = Image.new("RGB", src.size, (255, 255, 255))

    x0 = (src.width - obj.width) // 2 + dx
    y0 = (src.height - obj.height) // 2 + dy

    x0 = max(0, min(x0, src.width - obj.width))
    y0 = max(0, min(y0, src.height - obj.height))

    canvas.paste(obj, (x0, y0))
    return canvas


def rotate_on_white(img: Image.Image, angle: float = 3) -> Image.Image:
    """Поворачивает изображение на белом фоне."""
    return img.convert("RGB").rotate(
        angle,
        resample=Image.Resampling.BICUBIC,
        expand=False,
        fillcolor=(255, 255, 255),
    )


def make_augmented_versions(img: Image.Image) -> List[Tuple[str, Image.Image]]:
    """
    Создает несколько простых аугментаций для train:
    - зеркальное отражение;
    - небольшой сдвиг;
    - небольшой поворот.
    """
    result: List[Tuple[str, Image.Image]] = []
    result.append(("flip", ImageOps.mirror(img)))
```

```
    result.append(("shift", shift_image(img, dx=14, dy=-8)))
    result.append(("rot", rotate_on_white(img, angle=3)))
    return result


def split_train_test(paths: List[Path], test_ratio: float = TEST_RATIO) -> Tuple[List[Path],
List[Path]]:
    """Делит список путей на train/test."""
    shuffled = paths[:]
    random.shuffle(shuffled)

    test_count = max(1, int(round(len(shuffled) * test_ratio))) if len(shuffled) > 1 else 0
    test_paths = shuffled[:test_count]
    train_paths = shuffled[test_count:]

    if len(shuffled) == 1:
        train_paths = shuffled
        test_paths = []

    return train_paths, test_paths


def process_class(class_name: str) -> None:
    """Обрабатывает один класс целиком."""
    source_dir = RAW_ROOT / class_name
    images = list_images(source_dir)

    if not images:
        print(f"[ПРЕДУПРЕЖДЕНИЕ] Для класса '{class_name}' не найдено изображений в {source_dir}")
        return

    train_paths, test_paths = split_train_test(images)

    print(f"\nКласс: {class_name}")
    print(f"Всего изображений: {len(images)}")
    print(f"Train: {len(train_paths)} | Test: {len(test_paths)}")

    for idx, img_path in enumerate(train_paths, start=1):
        prepared = prepare_single_image(img_path)
        stem = img_path.stem
        out_path = OUTPUT_ROOT / "train" / class_name / f"{stem}.jpg"
        save_image(prepared, out_path)

        if CREATE_AUGMENTATIONS:
            for suffix, aug_img in make_augmented_versions(prepared):
                aug_path = OUTPUT_ROOT / "train" / class_name / f"{stem}_{suffix}.jpg"
                save_image(aug_img, aug_path)

        if idx % 10 == 0 or idx == len(train_paths):
            print(f"  train: обработано {idx}/{len(train_paths)}")

    for idx, img_path in enumerate(test_paths, start=1):
        prepared = prepare_single_image(img_path)
        stem = img_path.stem
        out_path = OUTPUT_ROOT / "test" / class_name / f"{stem}.jpg"
        save_image(prepared, out_path)

        if idx % 10 == 0 or idx == len(test_paths):
            print(f"  test : обработано {idx}/{len(test_paths)}")


def clean_output_root() -> None:
    """Создает базовую структуру папок результата."""
    for subset in ("train", "test"):
```

```
        for class_name in CLASS_NAMES:
            ensure_dir(OUTPUT_ROOT / subset / class_name)


def main() -> None:
    random.seed(RANDOM_SEED)

    print("Подготовка датасета начата...")
    print(f"Исходная папка: {RAW_ROOT.resolve()}")
    print(f"Выходная папка: {OUTPUT_ROOT.resolve()}")
    print(f"Классы: {CLASS_NAMES}")
    print(f"Размер итогового изображения: {OUTPUT_SIZE}x{OUTPUT_SIZE}")
    print(f"Порог фона: {BACKGROUND_THRESHOLD}")
    print(f"Padding объекта: {OBJECT_PADDING}")
    print(f"Доля test: {TEST_RATIO}")
    print(f"Аугментации train: {'включены' if CREATE_AUGMENTATIONS else 'выключены'}")

    clean_output_root()

    for class_name in CLASS_NAMES:
        process_class(class_name)

    print("\nГотово.")
    print("Подготовленный датасет сохранен в папке prepared_data")

if __name__ == "__main__":
    main()
```